

What the paper's missing

Three real, working capabilities shipped since the write-up was last touched — none of them are in the paper yet. This is the handoff: what's new, what needs writing, and exactly where the real numbers and commands live so you can check anything here yourself before it goes in.

Paper last edited 2026-09-13 Brief date 2026-09-15 Deadline 2026-09-20 AoE

Paper file `paper/mace_paper.tex`

01 WHAT'S NEW

Three commits' worth of real capability, each proven against actual hardware or a live LLM backend — not a mock, not a unit test standing in for a real run.

The `mace` CLI

NEW

A real interactive shell, modeled on Yosys/OpenROAD's own command style — `read_verilog`, `top_module`, `read_spec`, `set_core`, then `run`. State accumulates across commands the way a real EDA tool session does. Rich-rendered output: styled pass/fail, bordered panels for verification logs, a real command-reference table.

```
TRY IT mace shell --piton-root <path> --api <.env>
```

`mace/cli/shell.py` `mace/cli/session.py`

Vertex / Gemini backend

NEW

A second real LLM backend end to end, not just OpenCode — Google Vertex AI's Gemini models, authenticated via Application Default Credentials (no API key file at all, since ADC was already set up on this machine from earlier cluster work). Confirmed `gemini-2.5-flash` is what actually works on our project/region; `gemini-2.0-flash-001` 404s.

```
REAL RUN run_id f0ed1fe44745 · status=passed · 1x1 Ariane + barrier_atomic.c
```

`mace/llm.py` `chia.models.vertex.VertexGeminiLLM`

Verilator code coverage

NEW

Real line-coverage instrumentation, wired the whole way from a testbench patch to a live `run -coverage` command. Needed a genuine fix to OpenPiton's own hand-written testbench (it never called Verilator's coverage-write API), plus routing around a broken `verilator_coverage` install this project's own build environment resolves to first — both real, both documented in the script below.

```
REAL NUMBER 36.00% (8787/24311) · 1x1 Ariane + barrier_atomic.c, via the shell
```

```
scripts/local_coverage_1x1_build_test.py scripts/patch_openpiton.sh chia_openpiton/parse.py
```

02 WHAT THE PAPER NEEDS

Not prescribing your wording — just flagging where each capability plugs in. Pick what's worth the space given we're 5 days out.

☐ Mention the CLI as the real interface

The intro / system description currently describes the loop but not how someone actually drives it. One or two sentences plus the command example above is enough.

☐ Add Vertex as evidence of backend portability

Strengthens the "not tied to one LLM vendor" claim — worth a line in the evaluation or architecture section, with the real `run_id` above.

☐ Add coverage as a new verification axis

Pass/fail was the only signal before. A real 36.00% figure is worth a sentence in the evaluation table or a short new paragraph — your call on which fits better.

☐ Full read-through against current state

Standing item since the last status note — worth doing once the above lands, not before.

03 REFERENCE

Real paths and identifiers — check any claim above directly before it goes in the paper.

WHAT

WHERE

CLI source

`mace/cli/shell.py`

CLI docs	<code>docs/TECHNICAL_GUIDE.md §12</code>
----------	--

Coverage proof script	<code>scripts/local_coverage_1x1_build_test.py</code>
-----------------------	---

Coverage patch	<code>scripts/patch_openpiton.sh</code> , fix 5
----------------	---

Vertex run (no coverage)	<code>f0ed1fe44745</code>
--------------------------	---------------------------

Vertex + coverage run	<code>cb172af95c6b</code>
-----------------------	---------------------------

Metrics DB reader	<code>mace.metrics.summary()</code>
-------------------	-------------------------------------

Full commit list	<code>git log --oneline 0a58ac8..HEAD</code>
------------------	--

```
# reproduce the coverage number yourself
cd MACE
source ~/miniconda3/etc/profile.d/conda.sh && conda activate chia_env
bash scripts/patch_openpiton.sh /path/to/openpiton
python3 scripts/local_coverage_1x1_build_test.py
```

Questions on any number here → check the file/commit it points to first, it's all real and on `main`. **Nothing above is written into the paper yet.**